

Student Name	farjana.s
Register Number	TU-6253210211012
Project Title	High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch
Week	Week 4

WEEK-4 TASK

High-Performance Market Basket Analytics with Compressed FP-Trees from Scratch

1. Task Description

Students must develop a frequent pattern mining system from scratch to perform market basket analysis efficiently. The system should use a compressed FP-Tree structure to identify frequently occurring item combinations in a transactional dataset. It must implement dynamic tree structures, node-link pointers, header tables, conditional pattern bases, and recursively constructed conditional FP-Trees. The system should also generate high-confidence association rules while maintaining good execution performance for large datasets.

2. Objective

The main objective of this task is to develop an efficient market basket analysis system using the FP-Growth algorithm. The system should identify frequently purchased combinations of products and generate meaningful association rules without generating a large number of candidate itemsets. The implementation should focus on reducing execution time and memory usage through compressed tree structures and efficient data structures.

3. Key Requirements

1. Read and preprocess the transactional dataset.
2. Calculate the frequency and support of each item.
3. Apply a minimum support threshold to identify frequent items.
4. Construct a compressed FP-Tree dynamically.
5. Maintain node-link pointers between nodes containing the same item.
6. Create and maintain an FP-Tree header table.
7. Generate conditional pattern bases for frequent items.

8. Recursively construct conditional FP-Trees.
9. Mine frequent itemsets using the FP-Growth approach.
10. Generate association rules from the discovered frequent itemsets.
11. Calculate support, confidence, and lift for the generated rules.
12. Evaluate the execution time and scalability of the system.
13. Optimize the implementation for large transactional datasets.

4. Technologies and Concepts Used

- Python
- Data Structures
- FP-Growth Algorithm
- FP-Tree
- Header Table
- Node-Link Pointers
- Recursion
- Hash Tables and Dictionaries
- Association Rule Mining
- Support
- Confidence
- Lift
- Time Complexity
- Space Complexity

5. Proposed System Workflow

The system begins by loading a transactional dataset containing products purchased together by customers. The transactions are then cleaned and preprocessed by removing invalid values and standardizing item names.

Next, the frequency of every item is calculated. Items that do not satisfy the minimum support threshold are removed. The remaining frequent items are arranged in descending order of their frequency.

The ordered transactions are then inserted into a compressed FP-Tree. Common prefixes are shared between transactions, which reduces the amount of memory required to store the dataset.

A header table is maintained to store the frequency of each frequent item and the corresponding node-link pointers. These pointers connect nodes containing the same item and allow the mining process to access relevant nodes efficiently.

For each frequent item, a conditional pattern base is generated. The conditional pattern base is then used to construct a smaller conditional FP-Tree. This process is recursively repeated to discover frequent itemsets.

Finally, association rules are generated from the frequent itemsets. The system calculates support, confidence, and lift to identify strong and useful relationships between products.

6. Important FP-Tree Components

FP-Tree Node:

Each node stores an item name, item count, reference to its parent, references to its child nodes, and a node-link pointer.

Header Table:

The header table stores each frequent item, its total frequency, and the first node belonging to its node-link chain.

Node Link:

A node link connects different nodes containing the same item. This allows the mining algorithm to efficiently locate all occurrences of an item in the FP-Tree.

Conditional Pattern Base:

A conditional pattern base contains the prefix paths associated with a particular item. These paths are used to create a conditional FP-Tree.

Conditional FP-Tree:

A conditional FP-Tree is a smaller FP-Tree constructed from a conditional pattern base. It is used recursively to discover additional frequent patterns.

7. Evaluation Metrics

Support:

Support represents the percentage of transactions containing a particular itemset.

Support = Number of transactions containing the itemset / Total number of transactions

Confidence:

Confidence measures how frequently item B appears in transactions that already contain item A.

Confidence($A \rightarrow B$) = Support($A \cup B$) / Support(A)

Lift:

Lift measures the strength of an association between two itemsets compared with their independent occurrence.

Lift($A \rightarrow B$) = Confidence($A \rightarrow B$) / Support(B)

Execution Time:

The time required to construct the FP-Tree, mine frequent patterns, and generate association rules.

Memory Usage:

The amount of memory required for storing transactions, the FP-Tree, header table, conditional trees, and mined patterns.

Scalability:

The ability of the implementation to process increasingly large transactional datasets efficiently.

8. Expected Output

The completed system should produce:

1. A compressed FP-Tree representation.
2. An FP-Tree header table.
3. Node-link connections between identical items.
4. Frequent itemsets satisfying the minimum support threshold.
5. Conditional pattern bases.
6. Conditional FP-Trees.
7. Association rules satisfying the minimum confidence threshold.
8. Support values for frequent itemsets.
9. Confidence values for generated rules.
10. Lift values for generated rules.
11. Execution-time measurements.
12. Performance and scalability analysis.

9. Sample Association Rule

Consider the association rule:

{Bread, Butter} → {Milk}

This rule indicates that customers who purchase Bread and Butter may also have a strong tendency to purchase Milk.

The system should calculate the support, confidence, and lift of this rule and determine whether it satisfies the specified minimum confidence threshold.

10. Performance Optimization

The implementation should use efficient data structures such as hash maps or dictionaries for fast item-frequency calculations.

Node-link pointers should be used to avoid repeatedly scanning the complete FP-Tree. Infrequent items should be removed before constructing the tree to reduce its size.

Items should be arranged in descending order of frequency so that common prefixes can be shared and the FP-Tree can achieve better compression.

Conditional FP-Trees should only be constructed for relevant branches. The implementation should avoid unnecessary candidate generation and repeated scans of the complete transaction database.

The system should also be tested using datasets of different sizes to measure execution time, memory consumption, and scalability.

11. Conclusion

This task demonstrates the implementation of high-performance market basket analysis using the FP-Growth algorithm and compressed FP-Trees. By using dynamic tree structures, header tables, node-link pointers, conditional pattern bases, and recursively constructed conditional FP-Trees, the system can efficiently discover frequent purchasing patterns.

The generated association rules provide useful information about relationships between products. The use of support, confidence, and lift helps identify strong and meaningful associations. Overall, the task provides practical experience in advanced data structures, recursive algorithms, frequent pattern mining, and performance optimization.